Ansible ⭐ ⭐ — Interview-Ready Q&A

1. What is Ansible?
Ansible is an **automation and configuration management tool** used to automate tasks on servers and infrastructure.
It can be used for software installation, configuration changes, application deployment, service management, patching, and many other repetitive tasks.
Ansible is **agentless**, so normally we don't need to install an Ansible agent on the target Linux servers.
Simple ga gurthupettuko:
Ansible = Servers ni **automate + configure + deploy** cheyyadaniki tool.
Eng: Ansible is an agentless automation and configuration management tool used to configure servers, deploy applications, and automate repetitive operational tasks.

---

2. Why do we use Ansible?
Suppose we have 100 Linux servers and we need to install Nginx on all of them.
Manually login ayi install cheyyadam time-consuming and error-prone.
Ansible allows us to define the task once and execute it across multiple servers.
Simple ga gurthupettuko:
100 Servers → One Playbook → Automatic Configuration
Eng: Ansible is used to automate repetitive server administration and application deployment tasks consistently across multiple machines.

---

3. What is Ansible Inventory?
Inventory contains the list of servers that Ansible manages.
We can group servers based on their purpose or environment.
Example:
[webservers]
web01
web02

[appservers]
app01
app02

[dbservers]
db01
Simple ga gurthupettuko:
Inventory = **Ansible manage cheyyalsina servers list.**
Eng: Ansible Inventory defines the managed hosts and groups them logically so that automation can be targeted to specific systems.

---

4. What is a Playbook?
Ansible Playbook is a YAML file that defines the automation tasks we want Ansible to perform.
For example, a playbook can install packages, copy configuration files, start services, and deploy applications.
Simple ga gurthupettuko:
Playbook = **Automation instructions in YAML.**
Eng: An Ansible Playbook is a YAML-based automation definition that describes which tasks should be executed on which hosts.

---

5. What is a Task in Ansible?
A Task is a single unit of work performed by Ansible.
Examples:

- Install Nginx
- Copy a configuration file
- Start a service
- Create a user
- Restart an application

Multiple tasks can be grouped inside a Playbook.
Simple ga gurthupettuko:
Task = **One automation action.**
Eng: An Ansible task represents a specific action that Ansible performs on a managed host.

---

6. What is an Ansible Module?
Modules are the components Ansible uses to perform actual operations on target systems.
Examples include modules for package installation, file management, service management, users, and cloud resources.
For example:
```
ansible.builtin.package:
  name: nginx
  state: present
```
Simple ga gurthupettuko: Module = **Actual work perform chese component.**
Eng: Ansible modules provide the functionality used by tasks to perform operations such as package installation, file management, and service control.

7. What is the difference between Playbook and Module?
A **Playbook** defines the overall automation workflow.
A **Module** performs a specific operation.
For example, the Playbook may say "install and start Nginx," while package and service modules perform those individual actions.
Simple ga gurthupettuko:
Playbook = What steps to perform
Module = Actual operation perform chestundi
Eng: A Playbook defines the automation workflow, while modules perform the individual operations required by the tasks.

---

8. What is Ansible Role?
A Role is a standardized way to organize and reuse Ansible automation.
Instead of putting everything into one large Playbook, we can separate configuration into roles such as:
nginx-role
app-role
database-role
A Role can contain tasks, variables, templates, handlers, and other supporting files.
Simple ga gurthupettuko:
Role = **Reusable + Organized Ansible code.**
Eng: An Ansible Role is a reusable structure for organizing related automation tasks, variables, templates, handlers, and files.

---

9. Why do we use Roles?
In real projects, Playbooks can become very large.
Roles allow us to divide automation into reusable components and make the code easier to maintain.
For example, the same Nginx role can be reused across Dev, QA, and Production.
Simple ga gurthupettuko:
Large Playbook → Roles → Clean + Reusable Automation
Eng: Roles improve the maintainability, reusability, and organization of Ansible automation, especially in large projects.

---

10. What are Ansible Variables?
Variables are used to store values that can change between hosts or environments.
For example:
app_port: 8080
app_version: "2.0"
Instead of hardcoding values, we can use variables and provide different values for Dev, QA, and Production.
Simple ga gurthupettuko:
Variable = **Change ayye configuration value.**
Eng: Ansible variables allow us to parameterize automation so the same Playbook or Role can be reused with different environment-specific values.

---

11. What are Ansible Handlers?
Handlers are special tasks that run only when they are **notified by another task**.
They are commonly used for actions such as restarting or reloading services after a configuration change.
Example:
Config changed
↓
Notify Handler
↓
Restart Nginx
Simple ga gurthupettuko:
Handler = **Change jarigina appudu action perform chestundi.**
Eng: Ansible handlers are special tasks triggered by notifications, commonly used to restart or reload services when their configuration changes.

---

12. What is an Ansible Template?
Ansible templates are usually Jinja2-based files used to generate dynamic configuration files.
For example, instead of creating separate Nginx configuration files for Dev and Prod, we can use one template and populate values dynamically.
Simple ga gurthupettuko:
Template = **Dynamic configuration file.**
Eng: Ansible templates use Jinja2 expressions to generate configuration files dynamically based on variables.

---

13. What is Jinja2 in Ansible?
Jinja2 is the templating engine commonly used by Ansible.
It allows us to use variables, conditions, loops, and expressions inside templates.
Example:
server_port = {{ app_port }}
The value is populated when Ansible processes the template.
Simple ga gurthupettuko:
Jinja2 = **Template lo dynamic values use cheyyadaniki.**
Eng: Jinja2 is the templating engine used by Ansible to dynamically generate configuration files using variables and expressions.

14. What is Ansible Vault?
Ansible Vault is used to encrypt sensitive data used by Ansible.
Examples include:

- Passwords
- API credentials
- Secret variables
- Private configuration

This helps prevent sensitive information from being stored as plain text in repositories.
Simple ga gurthupettuko:
Vault = **Secrets ni encrypt cheyyadaniki.**
Eng: Ansible Vault provides encryption for sensitive Ansible data such as passwords and secret variables.

---

15. Why should we use Ansible Vault?
Suppose our Playbook needs a database password.
If we directly write the password inside the Playbook and commit it to Git, it becomes a security risk.
Using Ansible Vault, we can encrypt sensitive variables and provide controlled access to decrypt them during execution.
Simple ga gurthupettuko:
Plain-text password ❌
Encrypted secret with Vault ✅
Eng: Ansible Vault protects sensitive data from being exposed in source control and allows encrypted secrets to be used during automation.

---

16. Is Ansible agentless?
Yes, Ansible is generally **agentless**.
For Linux/Unix systems, Ansible commonly connects using SSH, while Windows systems can use Windows-specific communication mechanisms such as WinRM or supported connection methods.
No continuously running Ansible agent is normally required on the managed Linux server.
Simple ga gurthupettuko:
Ansible → SSH → Linux Server
No Ansible Agent required.
Eng: Ansible is agentless because it normally uses existing remote-management protocols such as SSH rather than requiring a dedicated agent on managed Linux hosts.

---

17. How does Ansible work?
The basic flow is:
Ansible Control Node
↓
Inventory
↓
Playbook
↓
SSH / Remote Connection
↓
Managed Servers
↓
Modules Execute Tasks
Ansible reads the inventory and Playbook, connects to the target hosts, and executes the required modules.
Simple ga gurthupettuko:
Control Node → Playbook → SSH → Server → Task
Eng: Ansible reads the inventory and Playbook, connects to managed hosts using the appropriate connection method, and executes modules to perform the requested tasks.

---

18. What is the Ansible Control Node?
The Control Node is the machine from which we run Ansible commands and Playbooks.
It contains Ansible and the required configuration, inventory, credentials, and automation code.
It communicates with the managed servers.
Simple ga gurthupettuko:
Control Node = **Ansible run chese machine.**
Eng: The Ansible Control Node is the system from which Ansible commands and Playbooks are executed against managed hosts.

---

19. What are Managed Nodes?
Managed Nodes are the servers or systems that Ansible configures and manages.
For example, EC2 instances running Linux applications can be managed nodes.
Ansible connects to these nodes and executes the required automation tasks.
Simple ga gurthupettuko:
Managed Node = **Ansible manage chese server.**
Eng: Managed Nodes are the target systems on which Ansible executes automation tasks.

20. What is Idempotency in Ansible?
Idempotency means running the same Ansible task multiple times should result in the desired state without unnecessarily changing an already-correct system.
For example, if Nginx is already installed and the task says state: present, Ansible should not reinstall it unnecessarily.
This makes automation safer and repeatable.
Simple ga gurthupettuko:
Already correct → No unnecessary change.
Eng: Idempotency means Ansible brings a system to the desired state without repeatedly making unnecessary changes when the desired state is already achieved.

---

21. What is ansible-playbook**?**
ansible-playbook is the command used to execute an Ansible Playbook.
Example:
ansible-playbook deploy.yml
We can also specify an inventory:
ansible-playbook -i inventory deploy.yml
Simple ga gurthupettuko:
ansible-playbook = Playbook execute cheyyadaniki command.
Eng: ansible-playbook is the command-line utility used to run Ansible Playbooks against the specified inventory.

---

22. What is Ansible Check Mode?
Check mode allows us to simulate what Ansible would change without actually applying those changes.
Example:
ansible-playbook deploy.yml --check
It is useful when we want to review potential changes before modifying production servers.
Simple ga gurthupettuko:
Check Mode = **What changes would happen ani preview.**
Eng: Ansible check mode performs a dry-run style execution to show potential changes without applying them where supported.

---

23. What is Ansible Ad-Hoc Command?
An ad-hoc command is used to perform a quick one-time task without creating a Playbook.
Example:
ansible all -m ping
It is useful for simple operations such as checking connectivity or executing a quick command.
Simple ga gurthupettuko:
Ad-Hoc = **Quick one-time task.**
Eng: Ansible ad-hoc commands are used for quick operational tasks without defining a complete Playbook.

---

24. How do you check whether Ansible can connect to servers?
I commonly use the Ansible ping module:
ansible all -m ping
If connectivity is successful, Ansible returns a successful response from the target host.
If it fails, I check SSH connectivity, credentials, inventory, network access, and host configuration.
Simple ga gurthupettuko:
ansible all -m ping = **Connectivity test.**
Eng: I use the Ansible ping module to verify connectivity between the control node and managed hosts, and troubleshoot authentication or network issues if it fails.

---

25. How do you run Ansible for a specific group?
We can specify the inventory group:
ansible-playbook -i inventory deploy.yml --limit webservers
This prevents the Playbook from running against every host in the inventory.
It is especially useful when we want to deploy only to a particular environment or server group.
Simple ga gurthupettuko:
--limit = **Specific servers/group only.**
Eng: I use --limit to restrict a Playbook execution to specific hosts or inventory groups.

---

26. How do you troubleshoot an Ansible Playbook failure?
First, I check the failed task and the error message.
Then I verify inventory, SSH connectivity, permissions, variables, module parameters, target-server state, and required dependencies.
I can also use verbose output such as:
ansible-playbook deploy.yml -vvv
to get more execution details.
Simple ga gurthupettuko:
Failed Task → Error → Connection → Variables → Permissions → Root Cause
Eng: I start with the failed task and error message, then verify connectivity, credentials, variables, module parameters, permissions, and target-server configuration.

27. What is the difference between Ansible and Terraform?
Terraform is mainly used for **provisioning infrastructure**.
Ansible is mainly used for **configuration management and application/server automation**.
For example:
Terraform → Create EC2 / VPC / RDS
Ansible → Configure EC2 / Install App / Start Service
They can be used together in a DevOps workflow.
Simple ga gurthupettuko:
Terraform = **Infrastructure create**
Ansible = **Server configure**
Eng: Terraform is primarily an Infrastructure as Code tool for provisioning infrastructure, while Ansible is primarily used for configuration management, deployment, and operational automation.

---

28. Can Terraform and Ansible be used together?
Yes.
For example, Terraform can create AWS infrastructure such as VPC, EC2, Security Groups, and Load Balancers.
After the EC2 instances are available, Ansible can connect to them and install applications, configure services, deploy files, and apply server configuration.
Simple ga gurthupettuko:
Terraform → Infrastructure
Ansible → Configuration
Eng: Terraform and Ansible can complement each other, with Terraform provisioning infrastructure and Ansible configuring and deploying software on the resulting systems.

---

29. Explain a real-time Ansible deployment.
Suppose a new application version needs to be deployed to 20 EC2 servers.
The application artifact is available, and Ansible inventory contains those servers.
The Playbook can copy the artifact, update configuration, stop or restart the required service, and perform validation.
Git
↓
Jenkins
↓
Build Application
↓
Artifact Repository
↓
Ansible Playbook
↓
20 EC2 Servers
↓
Deploy + Configure
↓
Restart Service
↓
Verify
Simple ga gurthupettuko:
Jenkins → Ansible → Multiple EC2 → Application Deployment
Eng: In a real-time project, Ansible can be integrated with Jenkins to automate application deployment and configuration across multiple EC2 instances consistently.

---

30. Why is Ansible useful in DevOps?
Ansible helps automate repetitive infrastructure and application operations.
It can be integrated with CI/CD tools such as Jenkins and used to configure servers, deploy applications, manage services, patch systems, and perform operational tasks.
Because the automation is defined as code, it is easier to standardize and version-control.
Simple ga gurthupettuko:
Manual Work → Ansible Automation → Fast + Consistent + Repeatable
Eng: Ansible is useful in DevOps because it automates configuration, deployment, and operational tasks consistently across environments and integrates well with CI/CD pipelines.